

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: THỰC TẬP CƠ SỞ
MÃ HỌC PHẦN: INT13147**

**BÀI THỰC HÀNH 4.1
LẬP TRÌNH CLIENT-SERVER ĐỂ TRAO ĐỔI THÔNG TIN AN TOÀN**

Sinh viên thực hiện:

B22DCAT107 Nguyễn Đức Hải

Giảng viên hướng dẫn: PGS. TS. Hoàng Xuân Dậu

HỌC KỲ 2 NĂM HỌC 2024-2025

MỤC LỤC

MỤC LỤC.....	2
DANH MỤC CÁC HÌNH VẼ.....	3
DANH MỤC CÁC TỪ VIẾT TẮT.....	4
CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH	5
1.1 Mục đích.....	5
1.2 Tìm hiểu lý thuyết	5
1.2.1 Socket là gì?	5
1.2.2 Socket hoạt động như thế nào?.....	5
1.2.3 Lập trình TCP Socket.....	6
CHƯƠNG 2. NỘI DUNG THỰC HÀNH	7
2.1 Chuẩn bị môi trường	7
2.2 Các bước thực hiện.....	7
2.2.1 Lập trình client và server với TCP socket.....	7
2.2.2 Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi.....	10
TÀI LIỆU THAM KHẢO	16

DANH MỤC CÁC HÌNH VẼ

Hình 1 – Tương tác TCP Socket giữa Client và Server	6
Hình 2 – Lập trình Client	7
Hình 3 – Lập trình Server.....	7
Hình 4 – Kết quả chạy trên Client.....	8
Hình 5 – Kết quả chạy trên Server	8
Hình 6 – Wireshark bắt gói tin gửi từ client đến server	9
Hình 7 - Wireshark bắt gói tin gửi từ server đến client.....	9
Hình 8 – Sửa code server	10
Hình 9 – Sửa code client	10
Hình 10 – Kết quả chạy code client	11
Hình 11 – Kết quả chạy code server	11
Hình 12 – Wireshark bắt gói tin gửi từ client đến server	12
Hình 13 - Wireshark bắt gói tin gửi từ server đến client.....	12
Hình 14 – Thay đổi giá trị key của client	13
Hình 15 – Kết quả chạy code trên server	13
Hình 16 – Kết quả chạy code trên client	14
Hình 17 – Wireshark bắt gói tin gửi từ client đến server	14
Hình 18 - Wireshark bắt gói tin gửi từ server đến client.....	15

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
TCP	Transmission Control Protocol	Giao thức điều khiển truyền
IP	Internet Protocol	Giao thức Internet
UDP	User Datagram Protocol	Giao thức gói dữ liệu người dùng

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH

1.1 Mục đích

Mục đích của bài thực hành “4.1. Lập trình client-server để trao đổi thông tin an toàn” là giúp sinh viên hiểu về cơ chế client/server và có thể tự lập trình client/server dựa trên socket, sau đó thực hiện ca đặt giao thức đơn giản để trao đổi thông tin an toàn.

1.2 Tìm hiểu lý thuyết

1.2.1 Socket là gì?

Socket là điểm cuối end-point trong liên kết truyền thông hai chiều (two-way communication) biểu diễn kết nối giữa Client – Server. Các lớp Socket được ràng buộc với một cổng port (thể hiện là một con số cụ thể) để các tầng TCP (TCP Layer) có thể định danh ứng dụng mà dữ liệu sẽ được gửi tới.

Socket là giao diện lập trình ứng dụng mạng được dùng để truyền và nhận dữ liệu trên internet. Giữa hai chương trình chạy trên mạng cần có một liên kết giao tiếp hai chiều, hay còn gọi là two-way communication để kết nối 2 process trò chuyện với nhau. Điểm cuối (endpoint) của liên kết này được gọi là socket.

Một chức năng khác của socket là giúp các tầng TCP hoặc TCP Layer định danh ứng dụng mà dữ liệu sẽ được gửi tới thông qua sự ràng buộc với một cổng port (thể hiện là một con số cụ thể), từ đó tiến hành kết nối giữa client và server.

Socket được chia chủ yếu thành 2 loại:

- Stream socket: Dựa trên giao thức TCP, chỉ thực hiện trên 2 tiến trình đã thiết lập kết nối. Giao thức này đảm bảo dữ liệu truyền đến nơi nhận 1 cách tin cậy và đúng tuần tự. Còn được gọi là socket hướng kết nối.
- Datagram socket: Dựa trên giao thức UDP, không yêu cầu thiết lập kết nối. Giao thức này không đảm bảo dữ liệu truyền đến nơi nhận 1 cách tin cậy và toàn vẹn. Còn được gọi là socket hướng không kết nối.

1.2.2 Socket hoạt động như thế nào?

Chức năng của socket là kết nối giữa client và server thông qua TCP/IP và UDP để truyền và nhận dữ liệu qua Internet. Giao diện lập trình ứng dụng mạng này chỉ có thể hoạt động khi đã có thông tin về thông số IP và số hiệu cổng của 2 ứng dụng cần trao đổi dữ liệu cho nhau. 2 ứng dụng cần truyền thông tin phải đáp ứng điều kiện sau thì socket mới có thể hoạt động:

- 2 ứng dụng có thể nằm cùng trên một máy hoặc 2 máy khác nhau
- Trong trường hợp 2 ứng dụng cùng nằm trên một máy, số hiệu cổng không được trùng nhau.

Ưu điểm lớn nhất của socket là hỗ trợ hầu hết các hệ điều hành bao gồm MS Windows, Linux... Ngoài ra, socket cũng được sử dụng với nhiều ngôn ngữ lập trình, gồm C, C++,

Java, Visual Basic, Visual C++... nên nó có thể tương thích với hầu hết mọi đối tượng người dùng với những cấu hình máy khác nhau.

Đặc biệt, người dùng cũng có thể chạy cùng một lúc nhiều socket liên tục, giúp nâng cao hiệu suất làm việc, cũng như tiết kiệm thêm nhiều thời gian và công sức hơn.

1.2.3 Lập trình TCP Socket

Các tiến trình mà muốn truyền thông với nhau thì sẽ gửi thông điệp thông qua các socket. Socket là cánh cửa của tiến trình ứng dụng và giao thức tầng transport (ở đây là TCP).

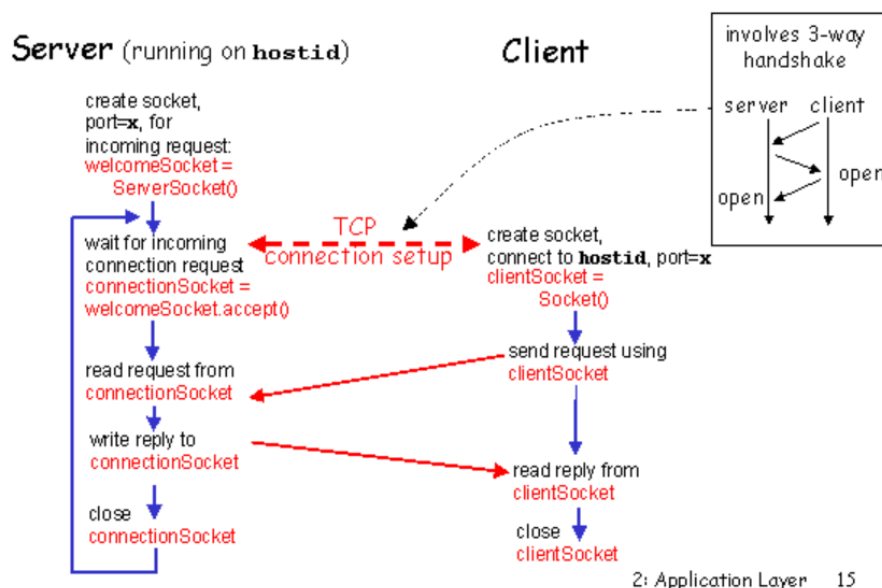
Để có thể tương tác với nhau và máy chủ có thể nhận liên lạc từ máy khách thì máy chủ phải luôn sẵn sàng. Điều này có 2 nghĩa, thứ nhất, giống như trường hợp của UDP, một tiến trình của máy chủ phải được chạy trước khi máy khách khởi tạo liên lạc đến. Thứ hai, chương trình chủ phải tạo ra 1 socket để sẵn sàng chấp nhận kết nối từ tiến trình khách.

Khi tiến trình chủ đã chạy, lúc này tiến trình khách sẽ tạo ra 1 socket TCP để có thể kết nối đến máy chủ. Trong khi máy khách đang tạo TCP socket, nó sẽ đặc tả địa chỉ IP, số cổng của tiến trình chủ.

Khi socket của tiến trình khách vừa được tạo, TCP trên máy khách sẽ tiến hành thực hiện quá trình bắt tay 3 bước và thiết lập kết nối TCP tới máy chủ.

Trong quá trình bắt tay 3 bước, khi tiến trình chủ nhận thấy tiến trình khách, nó sẽ tự tạo ra 1 socket mới chỉ dành riêng cho tiến trình khách đó. Khi được máy khách gõ cửa, chương trình kích hoạt với phương thức `accept()`. Cuối quá trình bắt tay 3 bước, một kết nối TCP tồn tại giữa socket của máy khách và socket của máy chủ.

Client/server socket interaction: TCP



Hình 1 – Tương tác TCP Socket giữa Client và Server

CHƯƠNG 2. NỘI DUNG THỰC HÀNH

2.1 Chuẩn bị môi trường

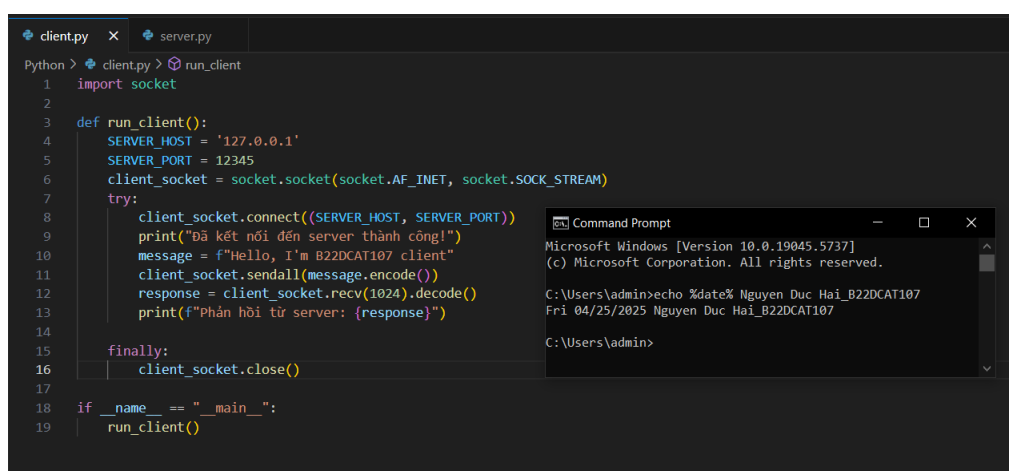
- Môi trường Python để chạy được ứng dụng client/server đã lập trình.
- Phần mềm Wireshark.

2.2 Các bước thực hiện

2.2.1 Lập trình client và server với TCP socket

▪ Lập trình client

Hàm `run_client()` tạo một socket TCP để kết nối tới server tại `127.0.0.1:12345`. Sau khi kết nối thành công, client gửi chuỗi “Hello, I'm B22DCAT107” tới server, rồi nhận phản hồi và in ra màn hình.

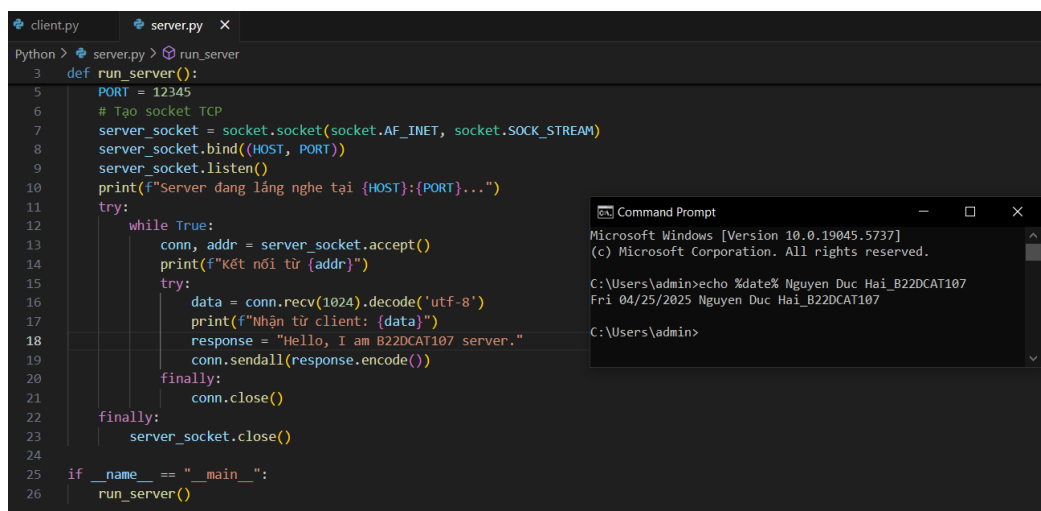


```
client.py x server.py
Python > client.py > run_client
1 import socket
2
3 def run_client():
4     SERVER_HOST = '127.0.0.1'
5     SERVER_PORT = 12345
6     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7     try:
8         client_socket.connect((SERVER_HOST, SERVER_PORT))
9         print("Đã kết nối đến server thành công!")
10        message = f"Hello, I'm B22DCAT107 client"
11        client_socket.sendall(message.encode())
12        response = client_socket.recv(1024).decode()
13        print(f"Phản hồi từ server: {response}")
14
15    finally:
16        client_socket.close()
17
18 if __name__ == "__main__":
19     run_client()
```

Hình 2 – Lập trình Client

▪ Lập trình server

Hàm `run_server()` tạo một socket TCP lắng nghe trên cổng 12345. Khi có client kết nối, server chấp nhận kết nối, nhận dữ liệu từ client, in ra dữ liệu nhận được, sau đó gửi lại phản hồi “Hello, I am B22DCAT107 server.”.



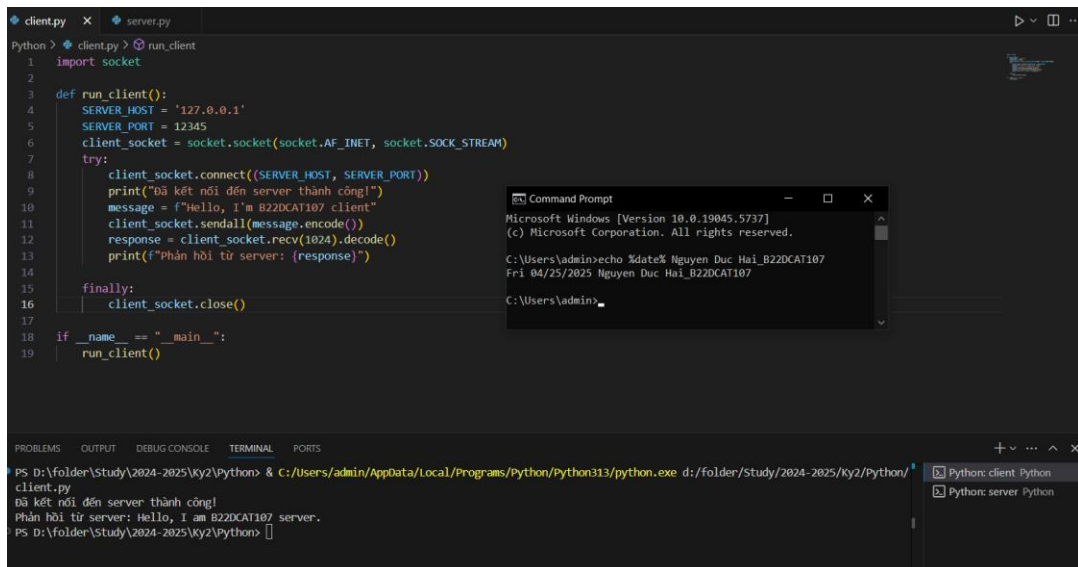
```
client.py x server.py
Python > server.py > run_server
3 def run_server():
4     PORT = 12345
5     # Tạo socket TCP
6     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7     server_socket.bind((HOST, PORT))
8     server_socket.listen()
9     print(f"Server đang lắng nghe tại {HOST}:{PORT}...")
10    try:
11        while True:
12            conn, addr = server_socket.accept()
13            print(f"Kết nối từ {addr}")
14            try:
15                data = conn.recv(1024).decode('utf-8')
16                print(f"Nhận từ client: {data}")
17                response = "Hello, I am B22DCAT107 server."
18                conn.sendall(response.encode())
19            finally:
20                conn.close()
21        finally:
22            server_socket.close()
23
24 if __name__ == "__main__":
25     run_server()
26
```

Hình 3 – Lập trình Server

- **Chạy server sau đó chạy client**

- Kết quả chạy trên Client.

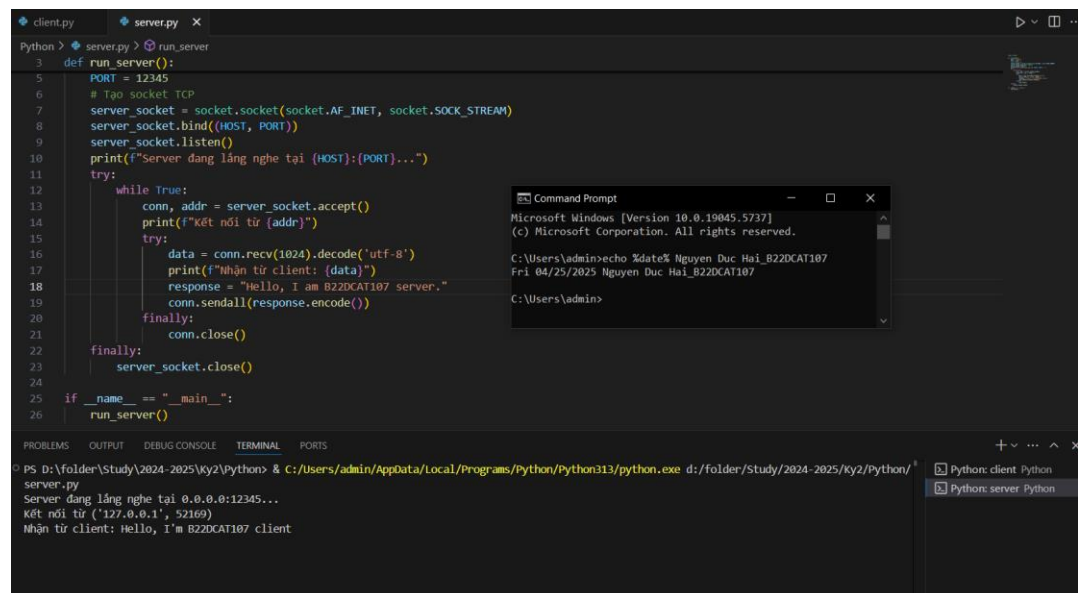
Client gửi thông điệp cá nhân hóa cho server: “Hello, I am B22DCAT107 client.”.



Hình 4 – Kết quả chạy trên Client

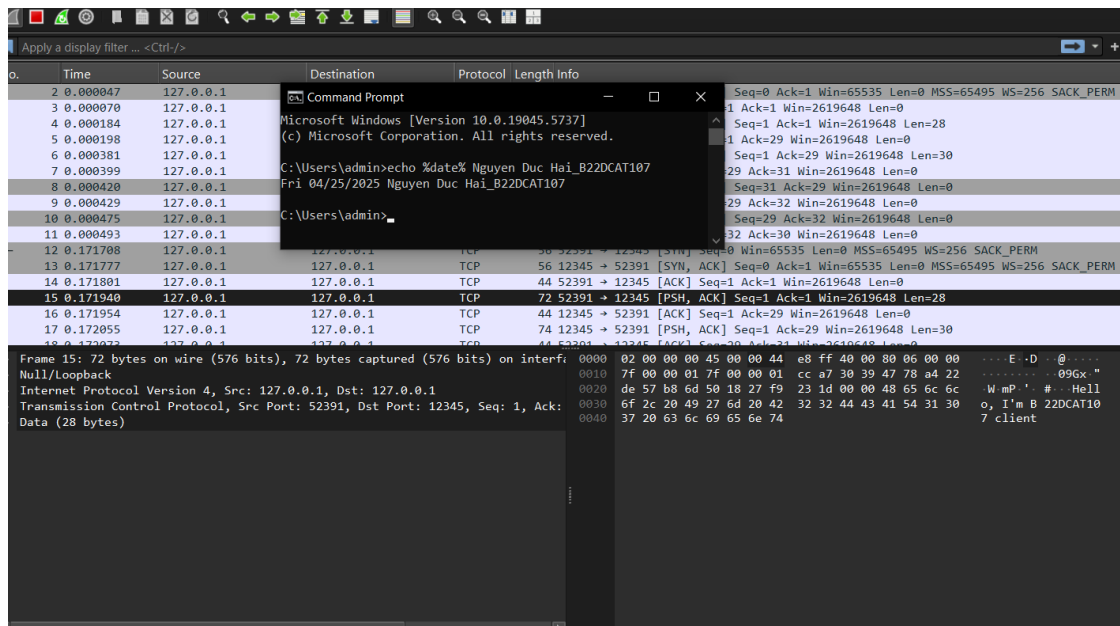
- Kết quả chạy trên Server.

Server nhận được hiển thị thông điệp nhận được và gửi lại client thông điệp: server gửi lại “Hello, I am B22DCAT107 server.”.



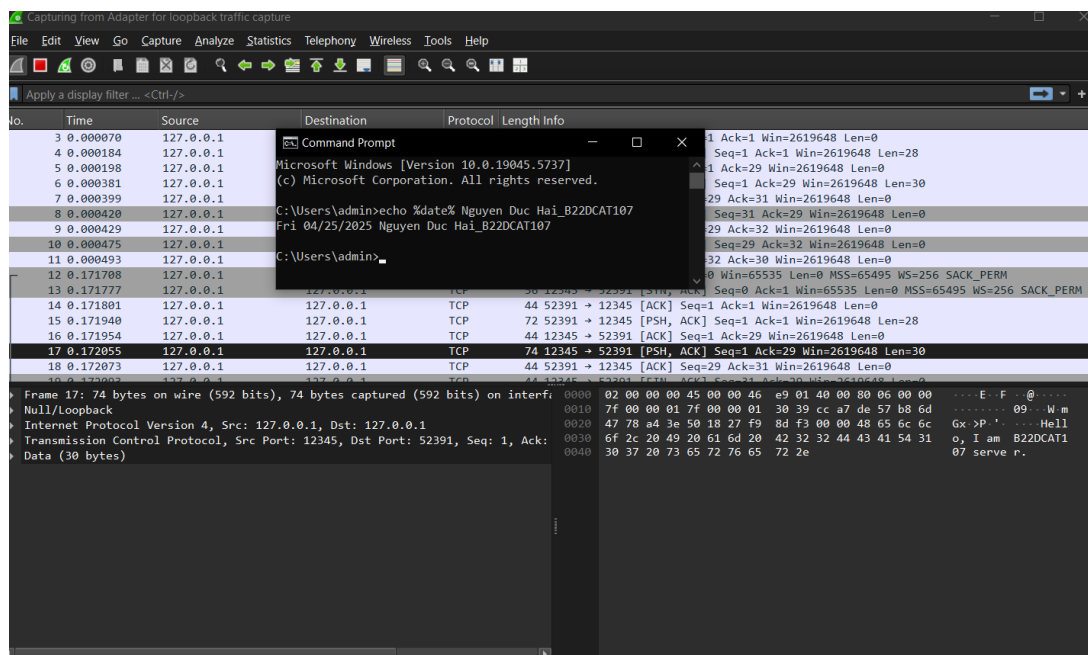
Hình 5 – Kết quả chạy trên Server

- **Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server và ngược lại**
 - Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server.



Hình 6 – Wireshark bắt gói tin gửi từ client đến server

- Sử dụng Wireshark để bắt các thông tin đã gửi từ server đến client.

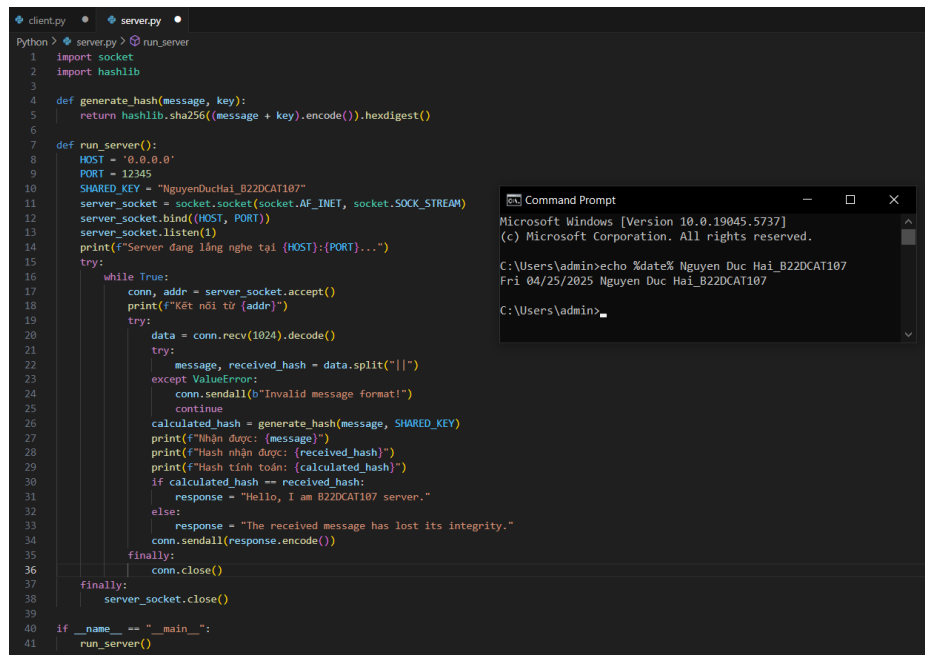


Hình 7 - Wireshark bắt gói tin gửi từ server đến client

2.2.2 Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi

▪ Sửa đổi code server

Hàm **run_server()** tạo socket TCP lắng nghe trên 0.0.0.0:12345 và dùng khóa chia sẻ **SHARED_KEY** để kiểm tra tính toàn vẹn dữ liệu. Khi client gửi dữ liệu (dạng **message||hash**), server tách ra, tính lại hash bằng SHA-256 sử dụng hàm **generate_hash()**, so sánh với hash nhận được. Nếu trùng khớp, server phản hồi “Hello, I am B22DCAT107 server.”, nếu không thì báo lỗi integrity.



```
client.py  server.py
Python > server.py > run_server
1 import socket
2 import hashlib
3
4 def generate_hash(message, key):
5     return hashlib.sha256((message + key).encode()).hexdigest()
6
7 def run_server():
8     HOST = '0.0.0.0'
9     PORT = 12345
10    SHARED_KEY = "NguyenDucHai_B22DCAT107"
11    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12    server_socket.bind((HOST, PORT))
13    server_socket.listen(1)
14    print(f'Server đang lắng nghe tại {HOST}:{PORT}...')
15    try:
16        while True:
17            conn, addr = server_socket.accept()
18            print(f'Kết nối từ {addr}:')
19            try:
20                data = conn.recv(1024).decode()
21                try:
22                    message, received_hash = data.split("||")
23                except ValueError:
24                    conn.sendall(b'Invalid message format!')
25                    continue
26                calculated_hash = generate_hash(message, SHARED_KEY)
27                print(f'Nhận được: {message}')
28                print(f'Hash nhận được: {received_hash}')
29                print(f'Hash tính toán: {calculated_hash}')
30                if calculated_hash == received_hash:
31                    response = "Hello, I am B22DCAT107 server."
32                else:
33                    response = "The received message has lost its integrity."
34                conn.sendall(response.encode())
35            finally:
36                conn.close()
37    finally:
38        server_socket.close()
39
40 if __name__ == "__main__":
41     run_server()
```

Command Prompt

Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

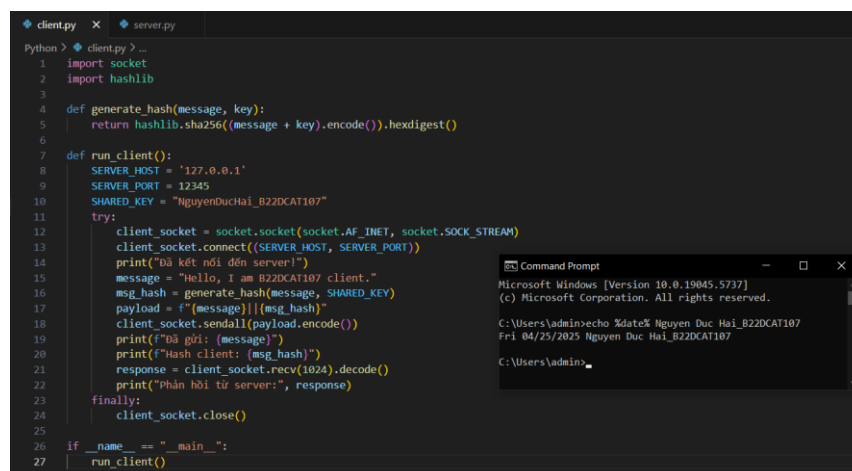
C:\Users\admin>echo %date% Nguyen Duc Hai_B22DCAT107
Fri 04/25/2025 Nguyen Duc Hai_B22DCAT107

C:\Users\admin>

Hình 8 – Sửa code server

▪ Sửa đổi code client

Hàm **run_client()** tạo socket TCP để kết nối tới server 127.0.0.1:12345. Client gửi chuỗi “Hello, I am B22DCAT107 client” kèm theo một mã băm SHA-256 sử dụng khóa bí mật **SHARED_KEY** tạo từ hàm **generate_hash()**. Dữ liệu gửi đi có dạng (**message||hash**). Sau khi gửi, client nhận phản hồi từ server và in ra màn hình.



```
client.py  server.py
Python > client.py > ...
1 import socket
2 import hashlib
3
4 def generate_hash(message, key):
5     return hashlib.sha256((message + key).encode()).hexdigest()
6
7 def run_client():
8     SERVER_HOST = '127.0.0.1'
9     SERVER_PORT = 12345
10    SHARED_KEY = "NguyenDucHai_B22DCAT107"
11    try:
12        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13        client_socket.connect((SERVER_HOST, SERVER_PORT))
14        print("Đã kết nối đến server!")
15        message = "hello, I am B22DCAT107 client."
16        msg_hash = generate_hash(message, SHARED_KEY)
17        payload = f'{message}||{msg_hash}'
18        client_socket.sendall(payload.encode())
19        print(f'Dã gửi: {message}')
20        print(f'Hash client: {msg_hash}')
21        response = client_socket.recv(1024).decode()
22        print(f'Phản hồi từ server: {response}')
23    finally:
24        client_socket.close()
25
26 if __name__ == "__main__":
27     run_client()
```

Command Prompt

Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>echo %date% Nguyen Duc Hai_B22DCAT107
Fri 04/25/2025 Nguyen Duc Hai_B22DCAT107

C:\Users\admin>

Hình 9 – Sửa code client

- Chạy server sau đó chạy client
 - Kết quả chạy code client.

```

1 import socket
2 import hashlib
3
4 def generate_hash(message, key):
5     return hashlib.sha256((message + key).encode()).hexdigest()
6
7 def run_client():
8     SERVER_HOST = '127.0.0.1'
9     SERVER_PORT = 12345
10    SHARED_KEY = "NguyenDucHai_B22DCAT107"
11    try:
12        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13        client_socket.connect((SERVER_HOST, SERVER_PORT))
14        print("Đã kết nối đến server!")
15        message = "Hello, I am B22DCAT107 client."
16        msg_hash = generate_hash(message, SHARED_KEY)
17        payload = f"{message}||{msg_hash}"
18        client_socket.sendall(payload.encode())
19        print(f"Đã gửi: {message}")
20        print(f"Hash client: {msg_hash}")
21        response = client_socket.recv(1024).decode()
22        print("Phản hồi từ server:", response)
23    finally:
24        client_socket.close()
25
26 if __name__ == "__main__":
27     run_client()
  
```

```

PS D:\folder\Study\2024-2025\ky2\Python> & c:/Users/admin/AppData/Local/Programs/Python/Python313/python.exe d:/folder/Study/2024-2025/ky2/Python/
client.py
client.py
Đã kết nối đến server!
Đã gửi: Hello, I am B22DCAT107 client.
Hash client: d909940dd79d48c0137ff7750ce32b815306605b060437d12bd30c2bf19d4dab
Phản hồi từ server: Hello, I am B22DCAT107 server.
  
```

Hình 10 – Kết quả chạy code client

- Kết quả chạy code server.

```

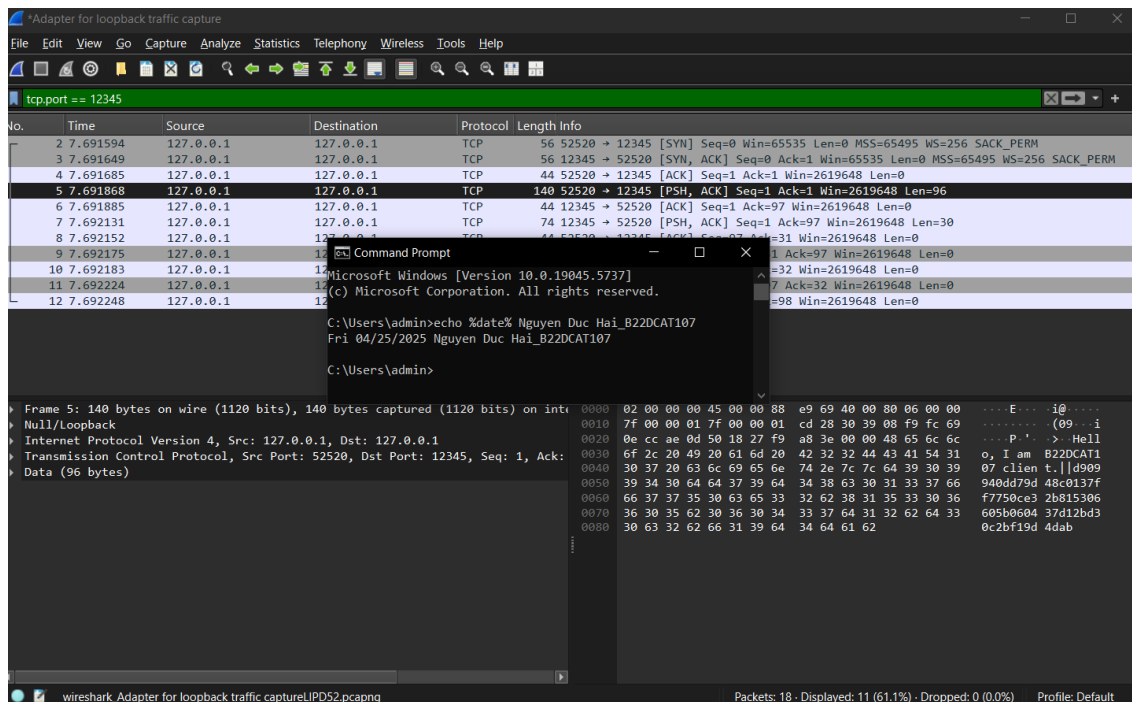
1 import socket
2 import hashlib
3
4 def generate_hash(message, key):
5     return hashlib.sha256((message + key).encode()).hexdigest()
6
7 def run_server():
8     HOST = '0.0.0.0'
9     PORT = 12345
10    SHARED_KEY = "NguyenDucHai_B22DCAT107"
11    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12    server_socket.bind((HOST, PORT))
13    server_socket.listen(1)
14    print(f"Server đang lắng nghe tại {HOST}:{PORT}...")
15    try:
16        while True:
17            conn, addr = server_socket.accept()
18            print(f"Kết nối từ {addr}")
19            try:
20                data = conn.recv(1024).decode()
21                try:
22                    message, received_hash = data.split("||")
23                except ValueError:
24                    conn.sendall(b"Invalid message format!")
25                    continue
26                calculated_hash = generate_hash(message, SHARED_KEY)
27                print(f"Nhận được: {message}")
28                print(f"Hash nhận được: {received_hash}")
  
```

```

PS D:\folder\Study\2024-2025\ky2\Python> & c:/Users/admin/AppData/Local/Programs/Python/Python313/python.exe d:/folder/Study/2024-2025/ky2/Python/
server.py
server.py
Server đang lắng nghe tại 0.0.0.0:12345...
Kết nối từ ('127.0.0.1', 52520)
Nhận được: Hello, I am B22DCAT107 client.
Hash nhận được: d909940dd79d48c0137ff7750ce32b815306605b060437d12bd30c2bf19d4dab
Hash tính toán: d909940dd79d48c0137ff7750ce32b815306605b060437d12bd30c2bf19d4dab
  
```

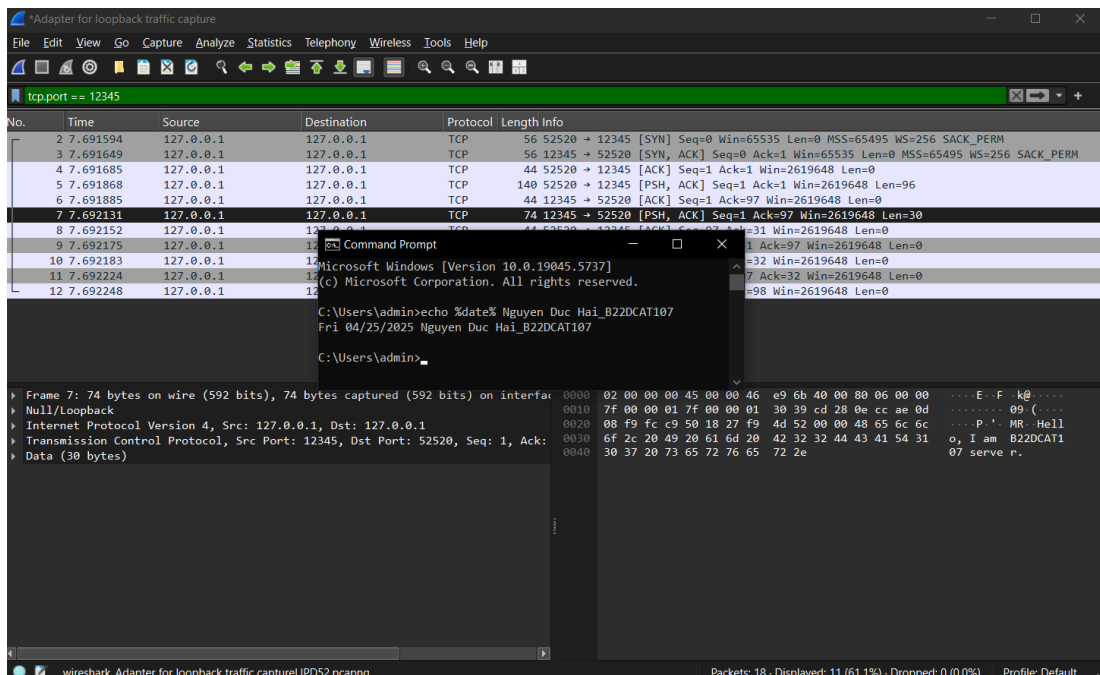
Hình 11 – Kết quả chạy code server

- **Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server và ngược lại**
 - Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server.



Hình 12 – Wireshark bắt gói tin gửi từ client đến server

- Sử dụng Wireshark để bắt các thông tin đã gửi từ server đến client.



Hình 13 - Wireshark bắt gói tin gửi từ server đến client

- Thay đổi giá trị key tại client và thực hiện gửi lại
 - Thay đổi giá trị key của client và giữ nguyên code của server.

```

client.py  X  server.py
Python > client.py > run_client
1  import socket
2  import hashlib
3
4  def generate_hash(message, key):
5      return hashlib.sha256((message + key).encode()).hexdigest()
6
7  def run_client():
8      SERVER_HOST = '127.0.0.1'
9      SERVER_PORT = 12345
10     SHARED_KEY = "B22DCAT107"
11     try:
12         client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13         client_socket.connect((SERVER_HOST, SERVER_PORT))
14         print("Đã kết nối đến server!")
15         message = "Hello, I am B22DCAT107 client."
16         msg_hash = generate_hash(message, SHARED_KEY)
17         payload = f"{message}||{msg_hash}"
18         client_socket.sendall(payload.encode())
19         print(f"Đã gửi: {message}")
20         print(f"Hash client: {msg_hash}")
21         response = client_socket.recv(1024).decode()
22         print("Phản hồi từ server:", response)
23     finally:
24         client_socket.close()
25
26 if __name__ == "__main__":
27     run_client()
  
```

Hình 14 – Thay đổi giá trị key của client

- Chạy server sau đó chạy client
 - Kết quả chạy trên server.

```

client.py  server.py  X
Python > server.py > run_server
1  import socket
2  import hashlib
3
4  def generate_hash(message, key):
5      return hashlib.sha256((message + key).encode()).hexdigest()
6
7  def run_server():
8      HOST = '0.0.0.0'
9      PORT = 12345
10     SHARED_KEY = "NguyenDucHai_B22DCAT107"
11     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12     server_socket.bind((HOST, PORT))
13     server_socket.listen(1)
14     print(f"Server đang lắng nghe tại {HOST}:{PORT}...")
15     try:
16         while True:
17             conn, addr = server_socket.accept()
18             print(f"Kết nối từ {addr}")
19             try:
20                 data = conn.recv(1024).decode()
21                 try:
22                     message, received_hash = data.split("||")
23                 except ValueError:
24                     conn.sendall(b"Invalid message format!")
25                     continue
26                 calculated_hash = generate_hash(message, SHARED_KEY)
27                 print(f"Nhận được: {message}")
28                 print(f"Hash nhận được: {received_hash}")
  
```

Hình 15 – Kết quả chạy code trên server

- Kết quả chạy trên client.

```

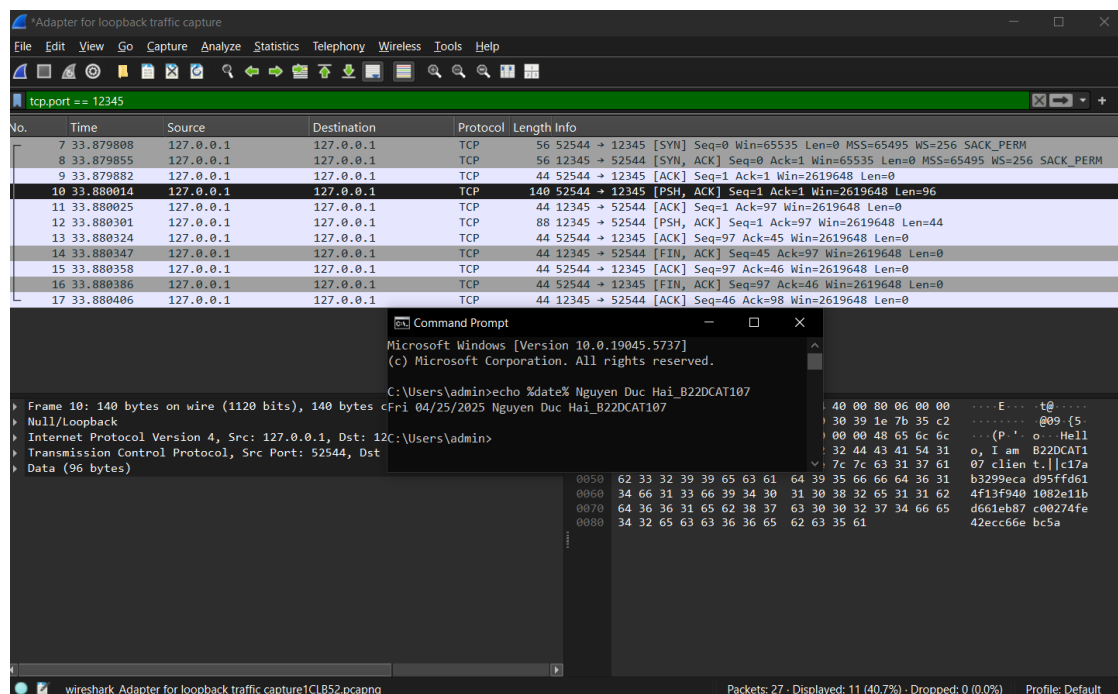
client.py x server.py
Python > client.py > run_client
1 import socket
2 import hashlib
3
4 def generate_hash(message, key):
5     return hashlib.sha256((message + key).encode()).hexdigest()
6
7 def run_client():
8     SERVER_HOST = '127.0.0.1'
9     SERVER_PORT = 12345
10    SHARED_KEY = "B22DCAT107"
11
12    try:
13        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14        client_socket.connect((SERVER_HOST, SERVER_PORT))
15        print("Đã kết nối đến server!")
16        message = "Hello, I am B22DCAT107 client."
17        msg_hash = generate_hash(message, SHARED_KEY)
18        payload = f"{message}||{msg_hash}"
19        client_socket.sendall(payload.encode())
20        print(f"Đã gửi: {message}")
21        print(f"Hash client: {msg_hash}")
22        response = client_socket.recv(1024).decode()
23        print("Phản hồi từ server:", response)
24    finally:
25        client_socket.close()
26
27 if __name__ == "__main__":
28     run_client()
  
```

```

PS D:\folder\Study\2024-2025\Ky2\Python> & C:/Users/admin/AppData/Local/Programs/Python/Python313/python.exe d:/folder/Study/2024-2025/Ky2/Python/client.py
Đã kết nối đến server!
Đã gửi: Hello, I am B22DCAT107 client.
Hash client: c17ab3299ecad95ff614f13f9401082e11bd661eb87c00274fe42ecc66ebc5a
Phản hồi từ server: The received message has lost its integrity.
PS D:\folder\Study\2024-2025\Ky2\Python>
  
```

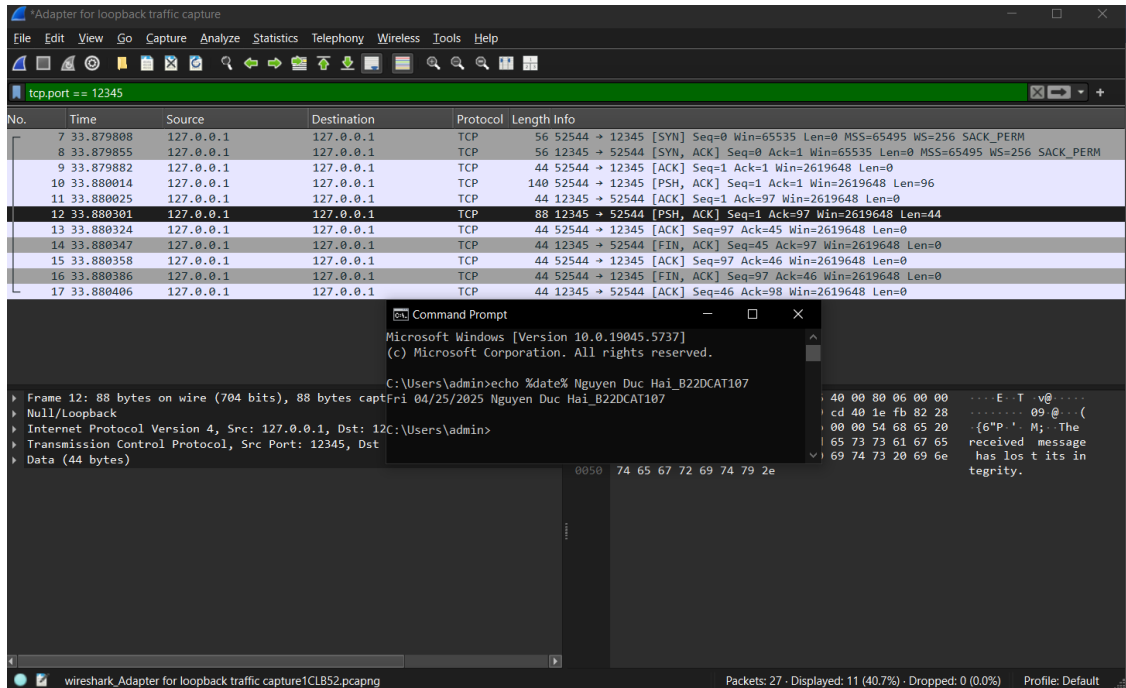
Hình 16 – Kết quả chạy code trên client

- ➔ Giá trị hàm băm bị thay đổi ➔ Không còn đảm bảo tính nguyên vẹn của thông điệp.
- Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server và ngược lại
- Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server.



Hình 17 – Wireshark bắt gói tin gửi từ client đến server

- Sử dụng Wireshark để bắt các thông tin đã gửi từ server đến client.



Hình 18 - Wireshark bắt gói tin gửi từ server đến client

TÀI LIỆU THAM KHẢO

- [1] Đinh Trường Duy, Phạm Hoàng Duy, Bài giảng Hệ điều hành Windows và Linux/Unix, Học viện Công Nghệ Bưu Chính Viễn Thông, 2022.
- [2] Tom Carpenter, Microsoft Windows Server Operating System Essentials, Sybex, 2011.
- [3] [Chapter 2: Application Layer V8.1 \(9/2020\)](#)
- [4] [Lập Trình Socket Với TCP/IP Trong Python](#)
- [5] [Socket là gì? Khái niệm cần biết về giao thức TCP/IP và UDP](#)